

PROMPT #1

PDF

NagySoft

Necesito que desgloses cada detalle de las cosas que encuentres en este archivo. hazlo de manera práctica y sencilla de comprender. Sin embargo, hay que trabajar de manera meticulosa, sin dejar detalles por fuera, porque



Basándonos rigurosamente en las especificaciones del documento oficial de **NagySoft**, aquí tienes un desglose meticuloso, estructurado por bloques de control y sin omitir ningún detalle para tu actividad evaluativa. PDF + 1

1. El Contexto y el Objetivo del Negocio

- **Tu Rol:** Eres un pasante seleccionado de la firma consultora NagySoft, cuyo dueño es el Dr. Béla Szilard. Los mejores pasantes reciben ofertas de empleo bien remuneradas. PDF + 1
- **Tu Supervisor Directo:** El VP de R&D (quien lidera el desarrollo de software, recluta talento y traduce los requerimientos del Dr. Szilard, ya que este último no habla español). PDF + 1
- **El Problema Principal:** Un reto de clasificación/agrupamiento en un espacio tridimensional (3D). Se te provee un archivo (`datos.csv`) donde cada línea representa un punto (x, y, z). PDF + 1
- **El Objetivo:** Identificar "grupos" (*clusters*) de puntos cercanos entre sí utilizando el

- El Objetivo: Identificar "grupos" (*clusters*) de puntos cercanos entre sí utilizando el algoritmo **k-means clustering**. PDF

:

2. Parámetros de Invocación y Consola

El programa debe ejecutarse estrictamente a través de la interfaz de línea de comandos (CLI) respetando estas directrices:

- **Nombre del Ejecutable:** Debe llamarse obligatoriamente `cluster` o `cluster.exe` . PDF
- **Formato de Invocación:** `cluster <k> <datos> .` PDF
 - `<k>` representa el número de clusters deseados. PDF
 - `<datos>` representa el nombre del archivo de entrada (ej: `datos.csv`). PDF

- o `<datos>` representa el nombre del archivo de entrada (ej: `datos.csv`). PDF

3. Restricciones Técnicas del Código (C++)

El Dr. Szilard y el VP de R&D son sumamente estrictos con las especificaciones del software:

PDF

- o **Lenguaje:** Implementado completamente en C++. PDF
- o **Modularidad Obligatoria:** El programa principal no debe centralizar las tareas, sino diferir a módulos específicos: PDF
 - o Un módulo separado exclusivamente para el cargador de datos. PDF
 - o Un módulo separado para el algoritmo de clustering. PDF
 - o Separación general recomendada entre procesos de entrada (carga) y de salida (escritura). PDF + 1
 - o Separación general recomendada entre procesos de entrada (carga) y de salida (escritura). PDF + 1
- o **Interfaces y Referencias:** Se deben utilizar interfaces y referencias para acceder a los módulos contenedores de la información (el documento pide guiarse por los ejemplos de la carpeta corporativa `C++/5.Tablas`). PDF + 1
- o **Condición de Parada:** Debe encapsularse obligatoriamente dentro de una **función**. Además, el código debe incluir un **comentario explícito** que explique detalladamente sus parámetros y justifique técnicamente por qué es una buena condición de parada. Puedes usar como guía el pseudocódigo pythonico de *Stanford CS221*. PDF + 3

4. Estructuras de Datos Requeridas

Debes declarar e implementar textualmente la siguiente estructura base para representar los puntos espaciales: PDF

```
C++
struct Coord_3D {
    double x;
    double y;
    double z;
};
```

(Nota: El documento original presenta los campos `double x;` `double y;` `double z;` inmediatamente debajo del cierre de la llave, pero lógicamente pertenecen al cuerpo de la estructura `Coord_3D`). PDF

- o **Regla de Inmutabilidad:** El cargador de datos debe volcar todos los puntos del archivo en un `vector<Coord_3D>` . Este vector **no debe ser modificado bajo ningún concepto** durante la ejecución del programa. PDF

5. Criterios de Experimentación y Control de "Overfitting"

- **Límite de Clusters:** El Dr. Szilard anticipa que **no vas a necesitar más de seis clusters** ($k \leq 6$). PDF
- **Nombres de los Clusters:** Cada grupo debe identificarse obligatoriamente con una sola letra mayúscula, utilizando las letras de la 'A' a la 'F'. PDF
- **El Peligro del Overfitting:** El documento prohíbe explícitamente añadir clusters de más con el único fin de inflar artificialmente la "perfección" del resultado. Se exige una clasificación honesta y con sentido común. PDF + 1
- **Métrica Estadística de Dispersión (Obligatorio):** Debes implementar una medida para evaluar la dispersión de cada cluster respecto a su propio centroide. Esta medida servirá para analizar matemáticamente en qué punto aumentar el valor de k deja de aportar mejoras significativas al agrupamiento. Puedes buscar una métrica adecuada en la literatura o diseñar una propia. PDF + 1

- **Estructura de Etiquetado (Opcional):** Se sugiere la estructura `Labeled` para asociar coordenadas con su cluster: PDF

C++

```
struct Labeled {  
    Coord_3D coord;  
    char label;  
};
```

El uso de esta estructura exacta no es mandatorio si tu algoritmo encuentra una vía más práctica, pero el mapeo final de los caracteres es obligatorio. PDF

2. `summary.txt`:

- Debe ser un archivo de resumen que siga estrictamente el siguiente formato de molde proporcionado por la empresa: PDF

Plaintext

```
A: N, (x, y, z), MD  
B: N, (x, y, z), MD  
C: N, (x, y, z), MD
```

- Donde `N` es la cantidad de puntos asignados a ese grupo, `(x, y, z)` representa las coordenadas del centroide calculado, y `MD` es el valor numérico de tu medida de dispersión. PDF
- **Regla de Criterio:** Aunque el molde de ejemplo muestra 3 clusters, tú debes entregar un **único archivo `summary.txt`** configurado con el número de clusters (k)

- Regla de Criterio: Aunque el molde de ejemplo muestra 3 clusters, tú debes entregar un único archivo `summary.txt` configurado con el número de clusters (k) que tú consideres técnicamente adecuado tras tu experimentación. PDF

7. Reglas de Integridad Académica y Uso de Herramientas

- **Recursos Permitidos:** Está totalmente permitido y alentado utilizar material de internet, libros, blogs y modelos de **Inteligencia Artificial**; su uso no afectará negativamente tu calificación. PDF + 1
- **Prohibición Absoluta:** Está terminantemente prohibido recibir asistencia de otras personas. El plagio entre compañeros resultará en la expulsión inmediata de la pasantía. PDF + 1
- **Requisito del Repositorio:** Es obligatorio que en tu repositorio crees una carpeta dedicada que contenga uno o más archivos donde: PDF
 1. Indiques las fuentes y materiales web consultados. PDF
 2. Muestres imágenes de soporte y reportes tus comentarios técnicos. PDF

+ Pregunta a Gemini

Flash



PROMPT #2:

CSV

datos

Okey, ya tengo una noción más clara. A continuación, te voy a proporcionar el documento requerido para comenzar el trabajo. Yo usaré VSCODE para compilar y te iré mostrando los resultados. Trabajaremos en conjunto, iremos paso a paso

¡Excelente! Es la mejor forma de avanzar. Trabajar de manera incremental y paso a paso en VS Code nos va a asegurar que cada módulo compile de forma perfecta y que se respeten minuciosamente todos los requerimientos que te acabo de desglosar. PDF + 1

minuciosamente todos los requerimientos que te acabo de desglosar. PDF + 1

Dado que ya tenemos el archivo `datos.csv` cargado en tu entorno, el **Paso 1** consiste en preparar la estructura de archivos en tu espacio de trabajo para garantizar la modularidad estricta exigida por la empresa. No pondremos todo en un solo archivo `.cpp`. PDF

Aquí tienes el orden lógico y la estructura de archivos que te propongo crear en tu carpeta de VS Code:

Estructura de Archivos Recomendada

```
Plaintext
MiProyectoNagySoft/
├── main.cpp           // Orquestador principal (valida los argumentos de la c
├── types.h           // Definición estricta de las estructuras de datos
├── DataLoader.h      // Interfaz del cargador de datos
├── DataLoader.cpp    // Implementación de la lectura del CSV
├── KMeans.h          // Interfaz del algoritmo y condición de parada
├── KMeans.cpp        // Implementación matemática de K-means
├── OutputHandler.h   // Interfaz para la exportación de archivos
└── OutputHandler.cpp // Implementación de clasificados.csv y summary.txt
```

PASO 2: El archivo de estructuras globales (`types.h`)

Este archivo define la forma de los datos tridimensionales obligatorios.

Crea un archivo nuevo llamado `types.h` y pega este código exacto:

C++



```
#ifndef TYPES_H
#define TYPES_H

// Estructura corporativa obligatoria para puntos en 3D
struct Coord_3D {
    double x;
    double y;
    double z;
};
```

```
double z;
};

// Estructura sugerida para manejar los puntos etiquetados
struct Labeled {
    Coord_3D coord;
    char label;
};

#endif // TYPES_H
```

PASO 3: El Módulo Cargador de Datos (`DataLoader.h` y `DataLoader.cpp`)

Este módulo se encarga puramente de leer el archivo `datos.csv` e introducirlo en el vector inmutable.

Crea un archivo nuevo llamado `DataLoader.h` y pega esto:

C++



```
#ifndef DATALOADER_H
#define DATALOADER_H

#include <vector>
#include <string>
#include "types.h"

// Firma del cargador modular por referencia
bool load_csv_data(const std::string& filename, std::vector<Coord_3D>& data_vector);

#endif // DATALOADER_H
```

Ahora, crea otro archivo llamado `DataLoader.cpp` y pega esto:

C++



```
#include "DataLoader.h"
#include <fstream>
#include <sstream>
#include <iostream>

bool load_csv_data(const std::string& filename, std::vector<Coord_3D>& data_vector) {
    std::ifstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Error corporativo: No se pudo abrir el archivo: " << filename << "\n";
        return false;
    }

    std::string line;
    while (std::getline(file, line)) {
        if (line.empty()) continue; // Evita líneas en blanco al final

        std::stringstream ss(line);
        std::string valX, valY, valZ;
```



```

// Procesa el formato separado por comas puro
if (std::getline(ss, valX, ',') &&
    std::getline(ss, valY, ',') &&
    std::getline(ss, valZ, ',')) {

    Coord_3D point;
    point.x = std::stod(valX);
    point.y = std::stod(valY);
    point.z = std::stod(valZ);

    data_vector.push_back(point);
}
}

file.close();
return true;
}

```

PASO 4: El Módulo Matemático del Algoritmo (`KMeans.h` y `KMeans.cpp`)

Aquí empaquetamos la lógica del K-Means, la medida de dispersión y la condición de parada súper documentada.

Crea un archivo nuevo llamado `KMeans.h` y pega esto:

```

C++

#ifndef KMEANS_H
#define KMEANS_H

#include <vector>
#include "types.h"

// Estructura interna para almacenar clusters
struct Cluster {
    Coord_3D centroid;
    std::vector<Coord_3D> points;
    double dispersion;
};

```

```

        double dispersion;
    };

    double calculate_distance(const Coord_3D& p1, const Coord_3D& p2);

    // Condición de parada encapsulada exigida por el Dr. Szilard
    bool should_stop(const std::vector<Coord_3D>& old_centroids,
                     const std::vector<Coord_3D>& new_centroids,
                     int current_iteration,
                     int max_iterations);

    std::vector<Cluster> run_kmeans(const std::vector<Coord_3D>& data, int k, int max_iter)

#endif // KMEANS_H

```

Ahora, crea un archivo llamado `KMeans.cpp` y pega esto:

C++

```

#include "KMeans.h"
#include <cmath>
#include <cstdlib>
#include <ctime>
#include <iostream>

double calculate_distance(const Coord_3D& p1, const Coord_3D& p2) {
    return std::sqrt(std::pow(p1.x - p2.x, 2) +
                     std::pow(p1.y - p2.y, 2) +
                     std::pow(p1.z - p2.z, 2));
}

/**
 * CONDICIÓN DE PARADA ENCAPSULADA
 * - Parámetros: Centroides viejos, nuevos, iteración actual y límite máximo.
 * - Justificación Técnica: Evalúa si el desplazamiento de los centros cayó por debajo
 * de un umbral de tolerancia mínimo (0.0001). Si ya no se mueven, continuar ejecutando
 * ciclos desperdiciaría CPU. Incluye un límite máximo por seguridad ante oscilaciones.
 */
bool should_stop(const std::vector<Coord_3D>& old_centroids,

```

```

    * ciclos desperdiciaría CPU. Incluye un límite máximo por seguridad ante oscilaciones.
    */
    bool should_stop(const std::vector<Coord_3D>& old_centroids,
                     const std::vector<Coord_3D>& new_centroids,
                     int current_iteration,
                     int max_iterations) {
        if (current_iteration >= max_iterations) return true;
        if (old_centroids.empty()) return false;

        double tolerance = 1e-4;
        for (size_t i = 0; i < old_centroids.size(); ++i) {
            if (calculate_distance(old_centroids[i], new_centroids[i]) > tolerance) {
                return false;
            }
        }
        return true;
    }

    std::vector<Cluster> run_kmeans(const std::vector<Coord_3D>& data, int k, int max_iterations) {
        int n = data.size();
        std::vector<Cluster> clusters(k);
        std::vector<Coord_3D> centroids(k);

        std::srand(std::time(0));

```

```

        std::srand(std::time(0));
        for (int i = 0; i < k; ++i) {
            centroids[i] = data[std::rand() % n];
        }

        int iteration = 0;
        std::vector<Coord_3D> old_centroids;

        while (!should_stop(old_centroids, centroids, iteration, max_iterations)) {
            old_centroids = centroids;

            for (int i = 0; i < k; ++i) {
                clusters[i].points.clear();
            }

            // Asignación de puntos
            for (const auto& point : data) {
                int best_cluster = 0;
                double min_dist = calculate_distance(point, centroids[0]);

                for (int i = 1; i < k; ++i) {
                    double dist = calculate_distance(point, centroids[i]);

```

```

        for (int i = 1; i < k; ++i) {
            double dist = calculate_distance(point, centroids[i]);
            if (dist < min_dist) {
                min_dist = dist;
                best_cluster = i;
            }
        }
        clusters[best_cluster].points.push_back(point);
    }
}

```

```

// Actualización de centroides
for (int i = 0; i < k; ++i) {
    if (clusters[i].points.empty()) continue;

    double sum_x = 0, sum_y = 0, sum_z = 0;
    for (const auto& p : clusters[i].points) {
        sum_x += p.x; sum_y += p.y; sum_z += p.z;
    }
    size_t num_points = clusters[i].points.size();
    centroids[i].x = sum_x / num_points;
    centroids[i].y = sum_y / num_points;
    centroids[i].z = sum_z / num_points;
}

```

```

    }
    size_t num_points = clusters[i].points.size();
    centroids[i].x = sum_x / num_points;
    centroids[i].y = sum_y / num_points;
    centroids[i].z = sum_z / num_points;
    clusters[i].centroid = centroids[i];
}

iteration++;
}

// Medida de dispersión estadística (MD) final
for (int i = 0; i < k; ++i) {
    if (clusters[i].points.empty()) {
        clusters[i].dispersion = 0.0;
        continue;
    }
    double total_dist = 0.0;
    for (const auto& p : clusters[i].points) {
        total_dist += calculate_distance(p, clusters[i].centroid);
    }
    clusters[i].dispersion = total_dist / clusters[i].points.size();
}
}

```

```

        double total_dist = 0.0;
        for (const auto& p : clusters[i].points) {
            total_dist += calculate_distance(p, clusters[i].centroid);
        }
        clusters[i].dispersion = total_dist / clusters[i].points.size();
    }

    std::cout << "Algoritmo finalizado de forma limpia en " << iteration << " iteraciones." << std::endl;
    return clusters;
}

```

PASO 5: El Módulo de Reportes de Salida (`OutputHandler.h` y `OutputHandler.cpp`)

Este archivo controla la escritura exacta en disco con el formato rígido del molde empresarial de NagySoft.

Crea un archivo nuevo llamado `OutputHandler.h` y pega esto:

C++

```

#ifndef OUTPUTHANDLER_H
#define OUTPUTHANDLER_H

#include <vector>
#include <string>
#include "KMeans.h"

bool save_classified_csv(const std::string& filename, const std::vector<Cluster>& clusters);
bool save_summary_txt(const std::string& filename, const std::vector<Cluster>& clusters);

#endif // OUTPUTHANDLER_H

```

Ahora, crea un archivo llamado `OutputHandler.cpp` y pega esto:

C++

C++

```
#include "OutputHandler.h"
#include <fstream>
#include <iostream>
#include <iomanip>

bool save_classified_csv(const std::string& filename, const std::vector<Cluster>& clusters) {
    std::ofstream file(filename);
    if (!file.is_open()) return false;

    char labels[] = {'A', 'B', 'C', 'D', 'E', 'F'};

    for (size_t i = 0; i < clusters.size(); ++i) {
        char current_label = labels[i % 6];
        for (const auto& point : clusters[i].points) {
            file << point.x << "," << point.y << "," << point.z << "," << current_label << "\n";
        }
    }
    file.close();
    return true;
}
```

```
    }
}
file.close();
return true;
}

bool save_summary_txt(const std::string& filename, const std::vector<Cluster>& clusters) {
    std::ofstream file(filename);
    if (!file.is_open()) return false;

    char labels[] = {'A', 'B', 'C', 'D', 'E', 'F'};
    file << std::fixed << std::setprecision(5);

    // Formato exacto de Molde Empresarial: Letra: N, (x, y, z), MD
    for (size_t i = 0; i < clusters.size(); ++i) {
        char current_label = labels[i % 6];
        size_t n_points = clusters[i].points.size();
        Coord_3D c = clusters[i].centroid;
        double md = clusters[i].dispersion;

        file << current_label << ": " << n_points << ", ("
            << c.x << ", " << c.y << ", " << c.z << ")", " << md << "\n";
    }
}
```

```

double md = clusters[i].dispersion,

    file << current_label << ": " << n_points << ", ("
        << c.x << ", " << c.y << ", " << c.z << ")", " << md << "\n";
}

file.close();
return true;
}

```

PASO 6: El Orquestador Principal (`main.cpp`)

El encargado de recibir los parámetros por consola y conectar los módulos.

Crea un archivo nuevo llamado `main.cpp` y pega esto:

```

C++

#include <iostream>
#include <vector>
#include <string>
#include <cstdlib>
#include "types.h"
#include "DataLoader.h"
#include "KMeans.h"
#include "OutputHandler.h"

int main(int argc, char* argv[]) {
    // Validación estricta de parámetros
    if (argc != 3) {
        std::cerr << "Uso reglamentario: " << argv[0] << " <k> <archivo_datos>" << s
        return 1;
    }

    int k = std::atoi(argv[1]);
    std::string filename = argv[2];

    if (k < 1 || k > 6) {

```

```

    }

    int k = std::atoi(argv[1]);
    std::string filename = argv[2];

    if (k < 1 || k > 6) {
        std::cerr << "Error: El numero de clusters k debe estar entre 1 y 6." << std::endl;
        return 1;
    }

    std::vector<Coord_3D> original_data;
    std::cout << "Iniciando carga de datos..." << std::endl;
    if (!load_csv_data(filename, original_data)) {
        return 1;
    }

    std::cout << "Dataset cargado con exito. Puntos leidos: " << original_data.size() << std::endl;

    std::cout << "Corriendo algoritmo clustering K-Means..." << std::endl;
    std::vector<Cluster> result_clusters = run_kmeans(original_data, k);

    std::cout << "Exportando artefactos de salida..." << std::endl;
    if (save_classified_csv("clasificados.csv", result_clusters) &&
        save_summary_txt("summary.txt", result_clusters)) {
        std::cout << ">> PROCESO CORPORATIVO EXITOSO. Archivos creados correctamente." << std::endl;
    }
}

```

```

    std::vector<Coord_3D> original_data;
    std::cout << "Iniciando carga de datos..." << std::endl;
    if (!load_csv_data(filename, original_data)) {
        return 1;
    }

    std::cout << "Dataset cargado con exito. Puntos leidos: " << original_data.size() << std::endl;

    std::cout << "Corriendo algoritmo clustering K-Means..." << std::endl;
    std::vector<Cluster> result_clusters = run_kmeans(original_data, k);

    std::cout << "Exportando artefactos de salida..." << std::endl;
    if (save_classified_csv("clasificados.csv", result_clusters) &&
        save_summary_txt("summary.txt", result_clusters)) {
        std::cout << ">> PROCESO CORPORATIVO EXITOSO. Archivos creados correctamente." << std::endl;
    } else {
        std::cerr << "Error critico al guardar los resultados." << std::endl;
        return 1;
    }

    return 0;
}

```


PASO 7: Mover el archivo de datos y Ejecutar

1. Copia tu archivo `datos.csv` y pégalo directamente dentro de esta nueva carpeta `Proyecto_NagySoft`.
2. En la terminal limpia de VS Code, ejecuta este comando para compilar de golpe de forma correcta:

PowerShell

```
g++ -std=c++11 main.cpp DataLoader.cpp KMeans.cpp OutputHandler.cpp -o cluster
```

3. Ejecuta el programa escribiendo:

PowerShell

```
.\cluster.exe 3 datos.csv
```

Finalmente:

Al compilar en VS CODE se obtuvo:

```
PS C:\Users\User\Desktop\Proyecto_Nagysoft> ls
```

Directorio: C:\Users\User\Desktop\Proyecto_Nagysoft

Mode	LastWriteTime	Length	Name
d----	21/06/2026 11:57 a. m.		.vscode
-a----	21/06/2026 12:26 p. m.	356265	cluster.exe
-a----	21/06/2026 12:23 p. m.	1054	DataLoader.cpp
-a----	21/06/2026 12:22 p. m.	262	DataLoader.h
-a----	20/06/2026 05:37 p. m.	110907	datos.csv
-a----	21/06/2026 12:21 p. m.	3630	KMeans.cpp
-a----	21/06/2026 12:21 p. m.	699	KMeans.h
-a----	21/06/2026 12:19 p. m.	1495	main.cpp
-a----	21/06/2026 12:19 p. m.	1396	OutputHandler.cpp

```
-a---- 21/06/2026 12:20 p. m.      325 OutputHandler.h
-a---- 21/06/2026 12:22 p. m.      304 types.h
```

```
PS C:\Users\User\Desktop\Proyecto_Nagysoft> g++ -std=c++11 main.cpp
DataLoader.cpp KMeans.cpp OutputHandler.cpp -o cluster
PS C:\Users\User\Desktop\Proyecto_Nagysoft>
```

```
PS C:\Users\User\Desktop\Proyecto_Nagysoft> .\cluster.exe 3 datos.csv
Iniciando carga de datos...
```

Dataset cargado con éxito. Puntos leídos: 4096

Corriendo algoritmo clustering K-Means...

Algoritmo finalizado de forma limpia en 31 iteraciones.

Exportando artefactos de salida...

>> PROCESO CORPORATIVO EXITOSO. Archivos creados correctamente.

```
PS C:\Users\User\Desktop\Proyecto_Nagysoft>
```

```
PS C:\Users\User\Desktop\Proyecto_Nagysoft> .\cluster.exe 3
datos.csv
Iniciando carga de datos...
Dataset cargado con éxito. Puntos leídos: 4096
Corriendo algoritmo clustering K-Means...
Algoritmo finalizado de forma limpia en 31 iteraciones.
Exportando artefactos de salida...
>> PROCESO CORPORATIVO EXITOSO. Archivos creados
correctamente.
PS C:\Users\User\Desktop\Proyecto_Nagysoft>
```

¡Sí! ¡Eso es todo! Mira la enorme diferencia ahora que todo está limpio desde cero:

1. **Puntos leídos: 4096** : Ahora sí leyó el dataset completo y real (no los 500 que tenías antes mezclados en la otra carpeta).
2. **31 iteraciones** : El algoritmo tuvo que trabajar de verdad y convergió de forma perfecta buscando el equilibrio matemático de esos 4096 puntos.

¡Sí! ¡Eso es todo! Mira la enorme diferencia ahora que todo está limpio desde cero:

1. `Puntos leídos: 4096` : Ahora sí leyó el dataset completo y real (no los 500 que tenías antes mezclados en la otra carpeta).
2. `31 iteraciones` : El algoritmo tuvo que trabajar de verdad y convergió de forma perfecta buscando el equilibrio matemático de esos 4096 puntos.
3. `PROCESO CORPORATIVO EXITOSO` : Tus módulos de lectura, lógica matemática y guardado se comunicaron sin romper nada.



